

Spring Boot 핵심 개념

IoC/DI/Bean, 계층 구조와 트랜잭션의 기본 원리를 학습합니다.

1. IoC, DI, Bean

Spring의 핵심은 객체 생성과 의존관계 관리를 애플리케이션 코드가 직접 담당하지 않고 Spring Container에 맡기는 것이다.

IoC(Inversion of Control)

객체의 생성, 연결, 생명주기 제어의 주도권이 개발자 코드에서 프레임워크로 넘어가는 것을 제어의 역전이라고 한다.

DI(Dependency Injection)

객체가 필요한 의존 객체를 내부에서 직접 생성하지 않고 외부에서 전달받는 방식이다. 생성자 주입은 필요한 의존성을 명확하게 표현하고 테스트하기 쉬워 일반적으로 선호된다.

Bean

Spring Container가 생성하고 관리하는 객체를 Bean이라고 한다. `@Component`, `@Service`, `@Repository`, `@Controller` 등으로 등록하거나 설정 클래스에서 `@Bean`으로 등록할 수 있다.

핵심 정리: IoC는 객체 제어권을 Container에 맡기는 개념이고, DI는 필요한 객체를 외부에서 주입받는 구현 방식이며, Bean은 Spring이 관리하는 객체이다.

학습 질문 예시: "DI가 뭐야?", "Spring Bean은 어떤 객체야?"

Spring Boot 핵심 개념

IoC/DI/Bean, 계층 구조와 트랜잭션의 기본 원리를 학습합니다.

2. 계층 구조와 트랜잭션

계층	주요 역할
Controller	HTTP 요청과 응답을 처리하고 입력을 Service에 전달
Service	비즈니스 규칙과 유스케이스를 처리
Repository	DB 저장과 조회 등 영속성 처리를 담당

트랜잭션(Transaction)

트랜잭션은 여러 DB 작업을 하나의 논리적 작업 단위로 묶는 개념이다. 모든 작업이 정상적으로 완료되면 Commit하고, 중간에 오류가 발생하면 Rollback하여 데이터 일관성을 지킨다.

Spring에서는 일반적으로 Service 계층의 비즈니스 작업에 `@Transactional`을 적용한다. 조회만 수행하는 경우 `readOnly=true`를 고려할 수 있다.

예를 들어 주문 생성 과정에서 주문 저장과 결제 정보 저장이 하나의 트랜잭션으로 묶여 있다면, 결제 정보 저장에 실패했을 때 주문 저장도 함께 롤백할 수 있다.

핵심 정리: Controller는 요청/응답, Service는 비즈니스 로직, Repository는 데이터 접근을 담당하며, 트랜잭션은 여러 DB 작업의 일관성을 보장한다.

학습 질문 예시: "Controller와 Service 역할 차이는?", "트랜잭션이 왜 필요해?"